CS 263 Final Project
Language Modeling with Neural Stochastic Differential Equations

Peter Racioppo

Spring 2023

Abstract

I examine directly predicting the next embedding in a sequence rather than predicting a probability distribution over tokens.

1 Introduction

Autoregressive language models trained on next-token prediction, such as GPT, have been extremely successful. However, predicting only a single token into the future limits their ability to plan or represent the long-term dynamics of text. Sampling is not differentiable, which means these models cannot be trained recurrently. This forces us to rely on reinforcement learning when training the model to perform goal-oriented generation.

For example, in reinforcement learning with human feedback (RLHF), a reward model trained from human evaluation of prompt-output pairs is used to train the language model as a reinforcement learning agent. (Leike et al., 2018) In practice, supervised learning is used in the first part of RLHF, but supervised learning is limited in that it compares tokens individually rather than the semantics of generated text. Reinforcement learning, in contrast, has the advantage that we can train a reward model that captures high-level semantic information about human preferences. However, the scalar reward provided in reinforcement learning is a much weaker signal than that provided by supervised learning. We would prefer to do supervised training of a classifier to predict whether generated text meets humans preferences, but sampling from a language model is not differentiable, and hence we cannot backpropagate from the classifier to the LM's weights. Hence, the limiting factor that forces us to rely on reinforcement learning rather than supervised learning is the next-token prediction loss.

If we could sample differentiably from a probability mass function (PMF), we could predict multiple tokens ahead in a recurrent manner and backpropagate the loss through time. There are several potential advantages to this approach. Predicting several tokens ahead rather than only the next token would force the language model to plan ahead during training, and would regularize the model by ruling out "short-sighted" models that can predict a single step ahead but diverge from the groundtruth over longer time horizons. This should improve model generalizability.

There is a method in the literature to sample from PMFs, using the Gumbel Softmax (Jang et al., 2017), (Maddison et al., 2017). Sampling y from a probability distribution softmax(a) is equivalent to setting $y =$argmax$(g + a)$, where g is i.i.d sampled from a Gumbel distribution. Each element in g can be computed as $g_i = -\log(-\log(u_i))$, where $u_i \sim \mathcal{U}(0, 1)$. We can approximate y as $\hat{y} =$softmax$((g + a)/\tau)$, $g \sim$Gumbel i.i.d. We now have an approximate one-hot encoding for the next token which we can differentiate through. In particular, the gradient is $\frac{\partial \hat{y}_i}{\partial a_j} = \hat{y}_i(\delta_{ij} - \hat{y}_j)/\tau$.

The main difficulty with this scheme is that because the one-hot encoding is only approximate, the resulting word embedding would not represent a discrete token, but rather a superposition of embeddings. We can expect that over several timesteps, the mixture of embeddings will smear out and may become unusable. (It's not entirely clear that this is the case however, since random vectors in a high-dimensional space are nearly orthogonal with high probability, so a mixture of several word embeddings will still be nearly orthogonal to most of the embeddings in the vocabulary.) The authors of (Gu et al., 2017) address this problem by applying the straight-through (non-differentiable) version of

the Gumbel-Softmax estimator to get a one-hot encoding, and then estimate the gradient update using the differentiable approximation. The latter is a biased approximation of the former, but the authors find that this works well in practice.

Another option would be to force the mixture of embeddings to more closely approach one of the word embeddings in the vocabulary. In particular, we could iteratively apply dot-product attention between the mixed word embedding and the embeddings of every token in the vocabulary, as in slot attention (Locatello et al., 2020). The mixed word embedding should rapidly approach one of the embeddings in the vocabulary.

More fundamentally, the difficulty of predicting multiple tokens ahead appears to be due to the need, at every time step, to translate between discrete tokens and continuous embeddings and then back to probabilities over discrete tokens. If the dynamics of text were deterministic, we could elide predicting next-token probabilities altogether and instead directly predict the next word embedding. Of course, natural language is stochastic and hence we need to be able to predict multiple possible next tokens. This can be done by introducing a latent variable. The latent variable can also encode things like goals. This approach also allows us to avoid predicting probabilities over every token in the vocabulary, which is surely expensive for a large vocabulary, and instead focus on learning the relationships between the most relevant embeddings.

2 Method

Let the ith embedding in a sequence be denoted x_i, and let us assume for now that we are dealing with a deterministic system. Given a sequence of embeddings $x_1, x_2, ..., x_i$, we wish to predict the next embedding x_{i+1}. Natural language is strongly non-Markov (the next embedding in a sequence may depend on embeddings many steps in the past). Hence, we must represent dynamics using a hidden state X_i, which may depend on all previous embeddings, $X_i = F(x_1, ..., x_i)$. Let us represent the dynamics of this phrase-level encoding as $X_{i+1} = g(X_i)$. To predict token x_{i+1}, we must decode using another mapping $x_{i+1} = f(X_i)$. Each of F, f, and g we represent with neural networks. Since F operates on a sequence, it can include attention, whereas f and g are fully-connected networks. To predict x_{i+1}, we can compute $x_{i+1} = f(X_i) = f(F(x_1, ..., x_i))$ and to predict x_{i+2}, we can compute $x_{i+2} = f(X_{i+1}) = f(g(X_i)) = f(g(F(x_1, ..., x_i)))$. Likewise, to predict x_{i+n}, we can compute $x_{i+n} = f(g^{n-1}(F(x_1, ..., x_i)))$. Updating the parameters requires backpropagating through time. The above method is illustrated in Fig. 1.

One possible problem with the foregoing method is that since F includes attention, this network is far more expressive than f and g. If we only predict a single time step ahead, F could, in principle, do all of the computation, while f and g are identity. However, predicting several steps ahead forces f and g to learn mappings that generalize well over several time steps, i.e. the dynamics. Hence, the learning objective should force the network to separate out state estimation (using attention) and dynamics. Another issue is that backpropagating through time significantly increases computation time and is limited by the exploding/vanishing gradient problem. Large language models are already quite coherent over fairly long time horizons, so it is not clear that predicting ahead several tokens would significantly improve performance. On the other hand, even predicting ahead several tokens may be sufficient to strongly regularize the model, and have significat effects when predicting over much longer time horizons.

In the more general case of stochastic dynamics, we assume that the dynamics of text can be modeled by a stochastic ordinary differential equation (or difference equation, actually) $X_{i+1} = Q(X_i, \eta)$, where η is a latent variable. Or, we could learn separate terms for the dynamics g and the noise q: $X_{i+1} = g(X_i) + q(X_i, \eta)$.

3 Experiments

I implemented the model in Fig. 1 in Pytorch, with the ability to predict up to five tokens in the future. I began by simulating a damped oscillator in 2D and experimented with using the model to learn the dynamics. The equations of motion are:
$\dot{x} = -0.1x + y$
$\dot{y} = -2x - 0.1y - 0.5xy - 0.025y^2$

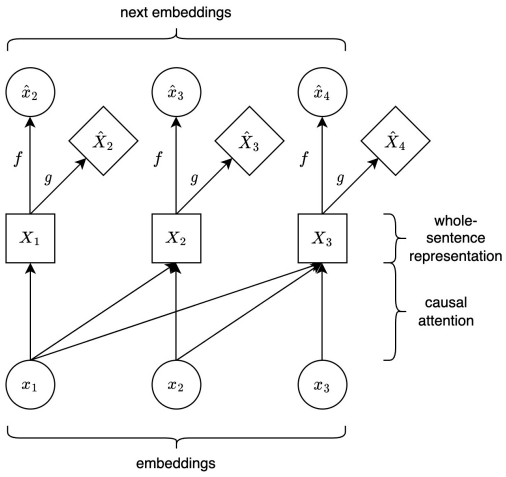

Figure 1: Illustration of the model. Causal attention maps from the input embeddings to a sentence-level representation, which is then decoded (with a neural network f) into the next-token predictions. A separate neural network learns a mapping g, which represents the dynamics of the sentence-level representation. At each time step, we predict not only the next token, but the next n tokens.

I added i.i.d. zero-mean Gaussian measurement noise to each coordinate at each time-step.

The model architecture is as described in the previous section, with F given by a one-layer self-attention module with a single head and a context window of 20 two-dimensional "tokens". We include a residual connection so that x_i is added directly to X_i. Here, f and g are two-layer fully-connected networks with linear activations followed by a multiplication node. This allows f and g to parsimoniously represent second order polynomials. Hence, we can easily determine whether f and g converge to the correct mappings by reading off the weights of the linear layers.

The expected behavior here is for f and g to converge to the same function $f = g = [\dot{x}, \dot{y}]$. In the case of zero measurement noise, the contribution from the self-attention layer should be zero, since the residual connection provides all the information needed to predict the next time step. However, if measurement noise is added to the data, we can do better by filtering the past position measurements to get an estimate of the actual current position.

It appears that predicting several time steps ahead does regularize the model and results in more accurate approximations of the dynamics. However, the contribution from the self-attention module effectively zeros out. That is, the model only attends to the previous time step, and effectively ignores all earlier time steps, even when the data is noisy. This confirms that f and g learn the right dynamics, but disproves my hypothesis that the attention module would learn how to filter the data. This may be because the attention module is not expressive enough to do effective state estimation.

4 Related Work

In (Chen et al., 2019), a method is introduced for synthesizing neural networks and differential equation solvers. (Jia and Benson, 2020) extends this method to ordinary differential equations with stochastic noise. (Lai et al., 2018) uses a CNN to learn short-term dependency patterns in time series data, along with an RNN to learn higher-level patterns.

Like the technique in this report, energy-based models avoid predicting probabilities over the entire state space, and instead make use of latent variables to enable neural networks to output multiple distinct values. (Deng et al., 2020) explores the use of energy-based models (EBMs) at the sequence level for text generation. "Time Control" models goal-less text as Brownian motion in a latent space. Goal-conditioned behavior is incorporated by conditioning on a fixed starting and/or ending point in this latent space. The latent space is learned with contrastive learning, and then used to improve local coherence. (Wang et al., 2023) Other important recent work in energy-based models includes (Zbontar et al., 2021) and (Bardes et al., 2022), which address the collapse problem suffered by energy-based models.

5 Conclusion and Future Work

We have shown that including a multi-step prediction in the training loss successfully regularizes our model and forces it to learn the correct dynamics of a simple deterministic system corrupted with measurement noise. While the dynamics are learned by the desired part of the network, we failed to show that our model could separately perform state estimation and dynamics estimation.

I did some preliminary work on simulating higher-dimensional dynamical systems and wrote my own dynamic simulation class so that I can efficiently simulate stochastic differential equations. My intention is to conclude the experiments in the dynamics setting and then extend the model to natural language.

References

Adrien Bardes, Jean Ponce, and Yann LeCun. 2022. Vicreg: Variance-invariance-covariance regularization for self-supervised learning.

Ricky T. Q. Chen, Yulia Rubanova, Jesse Bettencourt, and David Duvenaud. 2019. Neural ordinary differential equations.

Yuntian Deng, Anton Bakhtin, Myle Ott, Arthur Szlam, and Marc'Aurelio Ranzato. 2020. Residual energy-based models for text generation.

Jiatao Gu, Daniel Jiwoong Im, and Victor O. K. Li. 2017. Neural machine translation with gumbel-greedy decoding.

Eric Jang, Shixiang Gu, and Ben Poole. 2017. Categorical reparameterization with gumbel-softmax.

Junteng Jia and Austin R. Benson. 2020. Neural jump stochastic differential equations.

Guokun Lai, Wei-Cheng Chang, Yiming Yang, and Hanxiao Liu. 2018. Modeling long- and short-term temporal patterns with deep neural networks.

Jan Leike, David Krueger, Tom Everitt, Miljan Martic, Vishal Maini, and Shane Legg. 2018. Scalable agent alignment via reward modeling: a research direction.

Francesco Locatello, Dirk Weissenborn, Thomas Unterthiner, Aravindh Mahendran, Georg Heigold, Jakob Uszkoreit, Alexey Dosovitskiy, and Thomas Kipf. 2020. Object-centric learning with slot attention.

Chris J. Maddison, Andriy Mnih, and Yee Whye Teh. 2017. The concrete distribution: A continuous relaxation of discrete random variables.

Rose E Wang, Esin Durmus, Noah Goodman, and Tatsunori Hashimoto. 2023. Language modeling via stochastic processes.

Jure Zbontar, Li Jing, Ishan Misra, Yann LeCun, and Stéphane Deny. 2021. Barlow twins: Self-supervised learning via redundancy reduction.